

NIRIKSHAK (■■■■■■■■■■) / CHETAK

Strategic Evaluation, Architectural Reality Check & Investigation Reconstruction Blueprint

Target System: NIRIKSHAK (CHETAK Engine)	Document Version: 2.0 (Post-Critique Synthesis)
Core Focus: Investigation Reconstruction under Fragmented Telemetry	Axiom: Unusual Activity ≠ Confirmed Malicious Activity

1. Executive Verdict & The Reality Check

Verdict: Highly Viable, but Requires Critical Strategic Realignment.

The core concept of NIRIKSHAK—contextual risk scoring layered on top of authenticated access—addresses a legitimate, billion-dollar gap in cybersecurity (the foundational premise of User and Entity Behavior Analytics / Zero Trust continuous authorization). However, presenting the project as an *'AI Threat Detector that out-detects Splunk or Microsoft'* is an unforced error that hackathon judges immediately reject. The project must be strictly positioned as an **Investigation Decision-Support & Telemetry Reconstruction Tool**.

Evaluation Parameter	■ The Trap (Unwinnable Claim)	■ The Winning Angle (Grounded Pitch)
Core Claim	AI catches malicious insider threats automatically.	Reconstructs fragmented, multi-silo telemetry into a unified investigation narrative in seconds.
Role of AI / ML	AI model makes verdicts and convicts users.	Isolation Forest provides an objective 8% statistical outlier signal; it has no concept of intent.
Metric of Success	High detection accuracy on fake synthetic data.	Drastic reduction in Mean Time to Investigate (MTTI) from 45 minutes to 30 seconds.
Framing	Accusatory: 'Malicious user detected'.	Objective: 'Elevated investigation priority due to multi-source sequence anomaly. Human review required.'

2. Critical Critiques & Failure Risks

The architectural review identified four major risks that must be proactively mitigated in our implementation and presentation:

1. The Synthetic Data Trap:

You cannot generate synthetic data and claim: *'Therefore it works in national defense.'* Judges recognize that anyone can write synthetic data tailored to trigger their own algorithms. Synthetic data proves software pipelines work—it does not prove real-world defense accuracy.

2. Fake Precision (The 'Black Box Wearing a Number' Problem):

Outputting a score of 74.4 without raw contextual grounding looks like arbitrary mathematics. A score only achieves analyst trust when accompanied by explicit raw baselines: e.g., *'Accessed repository at 02:00 AM (typical: 09:00-18:00), transferred 1.8 GB (typical: 20 MB), device MAC never previously associated with actor.'*

3. Double-Counting Risk:

If an event originates from an unfamiliar device, this single root cause can trigger an 'Identity anomaly', a 'Device anomaly', and a 'Behavioral anomaly' simultaneously, artificially inflating the final risk score. Factors must be orthogonal or normalized.

4. ML Overclaiming:

Isolation Forest identifies geometric outliers in a normalized feature space. It cannot know human motivation or intent. An engineer fixing an urgent production outage at 2:00 AM is a statistical anomaly, but entirely legitimate.

3. The Solution: Investigation Reconstruction Under Fragmented Telemetry

Instead of designing NIRIKSHAK by asking *'What cool features should we add?'*, we anchor the system in an operational workflow chain:

THE OPERATIONAL ANCHORING CHAIN

What exact investigation problem exists? → Fragmented logs across disparate silos (Auth, Endpoint, Cloud, Network).

How is it currently solved? → Analysts spend 45+ minutes manually grepping timestamps, IP addresses, and usernames across SIEM tools.

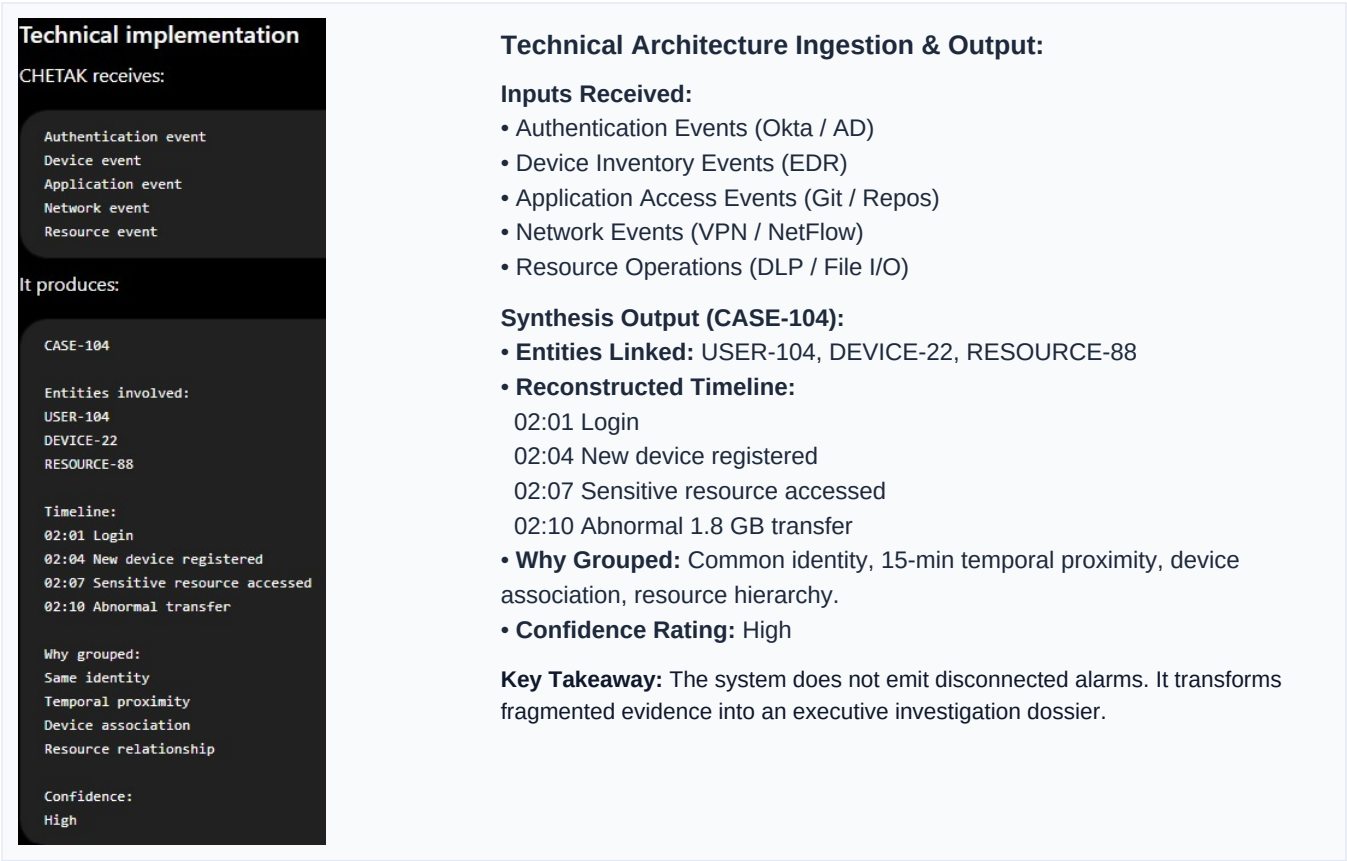
Where does the current workflow fail? → Slow correlation causes alert fatigue, missed dwell time, and unlinked multi-stage kill chains.

Can NIRIKSHAK improve that specific step? → Yes. It automatically groups related entities and builds a coherent investigation timeline in 5 seconds.

Can we measure the improvement? → Yes. By measuring Mean Time to Understand (MTTU) and Case Reconstruction Accuracy.

Figure 1: Technical Implementation Model — Case Reconstruction Dossier

NIRIKSHAK ingests multi-source telemetry and synthesizes a single bounded investigation hypothesis (Case Dossier) with timeline and grouping rationale.



4. The Three-Layer Evaluation Dataset Framework

To decisively defeat the '*synthetic data is fake*' objection during technical evaluation, NIRIKSHAK adopts a rigorous 3-layer validation strategy. This separates data that verifies code mechanics from data that scientifically tests real-world generalization.

Layer 1 — Controlled synthetic data

Use this for development.

Purpose:
Does the software work?

Example:

User login

+

New device

+

Sensitive access

+

Data transfer

This is useful for testing pipelines.

Layer 1: Controlled Synthetic Data

Purpose: Does the software pipeline work?
Used for unit and integration testing. Verifies that events flow through ingestion, scoring, and UI correctly.

Layer 3 — Scenario mutation

Take baseline scenarios and create variations.

For example:

Scenario A:
Obvious abnormal login

Then mutate:

Same working hours
Different resource

Same device
Different identity

Slow data transfer

Long time gaps

Missing events

Now CHETAK cannot simply memorize the demo pattern.

Layer 3: Scenario Mutation

Purpose: Does the engine memorize the demo pattern?
Adversarial perturbations: same hours + different resource, slow exfiltration, long time gaps, missing telemetry.

Layer 2 — Public cybersecurity datasets

Use realistic, established public datasets where legally and technically appropriate.

The purpose is:

Does CHETAK work on data we did not create ourselves?

This is much stronger.

Layer 2: Public Cybersecurity Datasets

Purpose: Does NIRIKSHAK work on data we did not create ourselves?
Validating against public benchmarks (e.g. CMU CERT Insider Threat dataset r4.2, LANL network logs, or BAF). This provides external scientific legitimacy and proves baseline detection is not hard-coded.

5. Data Normalization: The Canonical Event Schema

A critical flaw in naive security architectures is allowing source-specific log formats to penetrate the analytical engines. NIRIKSHAK strictly enforces a **Canonical Event Schema** through decoupled Ingestion Adapters:

3. Problem: Data normalization

This is solvable, but it must become a **core component**, not a preprocessing detail.

Solution: Canonical Event Schema

Do not allow every source to directly enter the intelligence engine.

Instead:

```
graph TD
    S1[SOURCE A] --> NL[NORMALIZATION LAYER]
    S2[SOURCE B] --> NL
    S3[SOURCE C] --> NL
    S4[SOURCE D] --> NL
    NL --> CE[CANONICAL EVENT]
    CE --> CE_E[CHETAK ENGINE]
```

Example canonical structure:

```
event_id
timestamp
source
event_type

actor_id
device_id
session_id
network_id

resource_id
action

metadata
```

The Normalization Principle:

Rule: NO SOURCE-SPECIFIC LOGIC INSIDE THE CORE ENGINE.

Every proprietary telemetry source is converted into a strongly typed Canonical Event via an adapter before analytical processing.

Canonical Event Fields:

- event_id: Unique UUIDv4
- timestamp: ISO 8601 UTC
- source: AUTH | ENDPOINT | APP | NETWORK
- event_type: Standardized operational verb
- actor_id: Normalized primary identity
- device_id: Hardware / MAC identifier
- session_id: Ephemeral session context
- network_id: IP / CIDR / subnet
- resource_id: Target entity / URI
- action: READ | WRITE | EXPORT | PRIV_ESC
- metadata: Extensible JSON payload

6. Architectural Solutions to Core Real-World Problems

The evaluation deep-dive addressed the specific technical hurdles required to make the prototype defensible:

Problem 4: Entity Resolution Across Silos

Users appear under different handles (e.g. aarya.m in Git, AARYA-M in Active Directory, EMP-1023 in HR).

Solution: Confidence-based probabilistic alias mapping. Rather than assuming identity, assign an Entity Match Confidence score (e.g. 0.85) based on email, device association, and session overlap.

Problem 5 & 6: Event Correlation & Over-Correlation

Simple rules like 'Same IP = Same Case' fail due to NAT and shared networks. Conversely, grouping everything into one case creates unmanageable noise.

Solution: Weighted Correlation Graph with adjustable confidence tiers: **Confirmed Group** (High confidence), **Candidate Related Events** (Medium), and **Contextual Background** (Low).

Problem 7: The Role of Artificial Intelligence

Letting an AI 'black box' score threats independently produces unpredictable hallucinations and destroys legal auditability.

Solution: Hybrid Risk Engine: Deterministic Rules (Identity, Device, Sensitivity, Baseline Deviations) form 92% of the decision weight; Isolation Forest provides an additive 8% anomaly signal.

Problem 9: Quantifying Analyst Time Savings

How do we prove NIRIKSHAK actually saves money and time?

Solution: Dual-arm benchmark experiment. Measure *Mean Time to Understand (MTTU)* and *Mean Time to Investigate (MTTI)* comparing raw terminal log grepping against NIRIKSHAK's timeline dossier.

Problem 14: Formal Definition of a 'Case'

A case is not an alert. **Case = A bounded investigation hypothesis.**

A case maintains a formal lifecycle (NEW → UNDER_REVIEW → RESOLVED / DISMISSED) and can only be transitioned by authenticated human security review.

Problem 15: Risk Score Fake Precision

A single number like 87.43 conveys false precision.

Solution: Explicitly decouple **Investigation Priority** (Low, Moderate, High, Critical) from **Confidence Level** (Low, Medium, High based on baseline sample depth).

7. Plain-Language Analogies (Explaining NIRIKSHAK Simply)

To communicate the technical essence of NIRIKSHAK clearly to any audience (judges, executives, or non-technical teammates), use these three intuitive real-world analogies:

ANALOGY 1: THE POLICE DETECTIVE (INVESTIGATION RECONSTRUCTION)

The Current SOC Challenge: Imagine a police detective investigating a bank break-in. Currently, the detective is handed 500 loose pieces of paper: 100 door-badge logs, 200 CCTV camera timestamps, and 200 computer logins—all in different folders. The detective spends 4 to 5 hours manually cross-referencing: *'Did Person A swipe in at 2:00, open computer B at 2:04, and access vault C at 2:08?'*

The NIRIKSHAK Advantage: We don't claim to build a 'magic robot that knows who is guilty'. Instead, NIRIKSHAK takes those 500 loose pages and automatically organizes them into a **single 4-step chronological timeline on one page** in 5 seconds. The human detective reviews the timeline and makes the decision in 30 seconds. **We eliminate the manual correlation bottleneck.**

ANALOGY 2: THE UNIVERSAL TRANSLATOR (CANONICAL EVENT SCHEMA)

The Multi-Log Chaos: Imagine 4 people in a room speaking 4 completely different languages: Windows logs speak French, network routers speak Japanese, Git logs speak Spanish, and AWS servers speak German. If your security code has to parse French, Japanese, Spanish, and German at every step, it becomes a tangled, buggy mess.

The Canonical Schema: Canonical schema acts as the **Universal Translator**. Before any log enters NIRIKSHAK's brain, an adapter translates it into standard English: *Who did it? From which device? At what time? What file? What action?* NIRIKSHAK's core engines only ever read clean, standardized English.

ANALOGY 3: THE 'LEGITIMATE-BUT-WEIRD' TEST (CONTEXTUAL SENSITIVITY)

The Midnight Bug Fix: Suppose a software engineer, Rahul, has an important product release tomorrow morning. At 2:30 AM, he logs in from home to fix an urgent bug.

- **Dumb Security Tool:** Screams **■ RED ALERT! ALIEN ATTACK! HACKER AT 2:30 AM! LOCK ACCOUNT!**
- **NIRIKSHAK:** Notes: *'Yes, 2:30 AM is off-hours. BUT Rahul is using his official corporate laptop, and he is touching his usual repository. Priority: MODERATE. Rationale: Likely working late before deadline. Analyst can dismiss with 1 click.'*

Why this wins: It proves to security judges that our platform understands context and does not cry wolf on innocent employees.

8. Summary: How Our Live Build Aligns

NIRIKSHAK's existing implementation on localhost:3001 directly realizes this architectural pivot:

- **Lean 3-tier Stack:** PostgreSQL 16 + FastAPI + Next.js (zero unneeded Redis/OpenSearch overhead).
- **Temporal Correlation Engine:** 15-minute sliding window evaluates sequences without black-box AI.
- **Multi-Stage APT Simulator:** Demonstrates external recon → lateral directory hop → privilege bypass → 1.8 GB bulk export.
- **Human-in-the-Loop Case Management:** Direct disposition actions (Dismiss / Escalate) backed by immutable audit trails.
- **Command Console:** Strictly an AST allowlist parser for rapid analyst triage—never an insecure OS shell.

End of Document — Prepared for the Hack2Ignite NIRIKSHAK Defense & Security Evaluation